一、为什么 PDF 不会重排

打开一本 EPUB，你可以随意改字号、行距、页边距，文字会重新流动，页码随之变化。而 PDF 不是这样：它把每一页的每一个字符、每一条线，都钉在一个以磅为单位的坐标系里。

一份 A4 页面是 595 乘 842 磅。这段文字之所以在现在的位置，是因为文件里写着「从 x 等于 62、y 等于 700 开始，用 11.5 磅的字号画出这几个字」。它没有「段落」的概念，只有一连串绘图指令。

所以 PDF 的字号是画出来的，不是排版出来的。你不能让一个已经画好的字变大而其余内容自动让位——那需要重新排版，而 PDF 从未保存排版所需的信息。

这也是为什么所有 PDF 阅读器的「夜间模式」，本质上都是对整页图像做颜色变换，而不是像 EPUB 那样换一套样式表。

二、固定版式的代价与好处

代价很直接：在小屏幕上，一页 A4 缩到手机宽度，正文可能只有原大的三分之一。你要么忍受小字，要么放大后左右平移。这是物理约束，不是实现偷懒。

好处同样直接：所见即所得。合同、论文、乐谱、画册、试卷，这些内容的价值恰恰在于版面本身。你不需要担心排版引擎理解错作者的意思。

一个务实的做法是：让读者在「整页」和「适应宽度」之间切换，把选择权交出去。默认适应宽度，需要看全貌时再缩回整页。

这条取舍没有标准答案，但它决定了阅读器的默认缩放策略，值得在产品层面明确写下来。

三、渲染管线

浏览器里渲染 PDF，通常借助 pdf.js：它把 PDF 的绘图指令解析出来，画到 canvas 上。你看到的每一页，其实是一张按屏幕像素密度生成的位图。

这意味着两件事。第一，缩放会触发重新渲染，因为位图放大会模糊；第二，渲染是异步的，快速翻页时必须取消上一帧，否则会出现两页重叠或卡顿。

高分屏上还要乘以设备像素比，否则文字边缘会发虚。但倍数不宜超过二，否则一张 A4 页面就要占用十几兆内存，连续翻几十页足以让标签页崩溃。

这些细节不会出现在需求文档里，却直接决定「读起来是不是舒服」。

四、进度与续读

EPUB 用 CFI 记录位置，因为它需要精确到某一段文字。PDF 简单得多：页码就是位置。第一页、第二页、第一百二十页，没有歧义。

但页码也有陷阱。如果文件被替换成另一个版本，页码可能指向完全不同的内容。稳妥的做法是同时记录总页数，页数对不上时提示读者重新定位。

另一个细节是「读到哪」的定义。是当前屏幕顶部的那一页，还是正在看的那一页？对单页模式来说两者相同；对连续滚动来说，前者更符合直觉。

本项目选择单页模式：一次只呈现一页，翻页即翻页。简单，也最接近纸书。

五、目录与书签

PDF 可以内嵌大纲（outline），结构类似树：章下面有节，节下面有小节。每个节点指向一个页面，也可能指向页面内的某个坐标。

解析大纲需要两步：先读出节点里的目标引用，再把引用换算成页码。前者是同步的，后者是异步的，所以整个目录的构建是一个递归的异步过程。

如果文件没有大纲，就只能靠页码导航。这时候进度条的价值会被放大——它是唯一能快速跳到「大概三分之一处」的控件。

书签则是另一回事：它记录的是读者自己认为重要的位置，和文件自带的结构无关。两者应该分开呈现，不要混在一个列表里。

六、边界情况

加密的 PDF 会要求密码。没有密码就无法解密内容流，连页数都读不出来。产品上应该明确提示「有密码保护」，而不是笼统地说「文件损坏」。

扫描件是另一类：整页只有一张图片，没有文字层。它能正常显示，但无法选中、无法搜索。如果将来要做全文搜索，必须先把这类文件排除，或者接入 OCR。

还有体积。一份带插图的论文可能有几十兆，导入时要给出进度反馈，读取时要用副本，避免原始数据被解析器转移后无法再次保存。

把这些边界写清楚，比多做一个功能更能减少日后的返工。